

Platform Architecture

Seven services, eight data domains, one metric definition. How Fraud360 ingests payments and core-banking events, scores them against each bank's own configuration, and carries a signal through to a defensible regulatory filing.

DOCUMENT HLD / FRMS-EWS	VERSION 1.8	DATE 2 October 2026	STACK Python · FastAPI · PostgreSQL
COMPANION BRD v1.9			

WHAT CHANGED SINCE V1.0

v1.0 described a platform with no authentication layer worth the name, no way to file a return, and eight detection indicators inert for want of data feeds. All three have been closed. This revision rewrites **identity and access** (§13) and **data protection** (§14) from scratch, adds **regulatory artefacts** (§10), **backfill, replay and simulation** (§8) and a seventh service, and restates the gap register against the R1–R5 release gates in BRD v1.9. Sections whose design has not changed are unchanged.

CONTENTS

1	Design principles	11	Regulatory artefacts
2	System context	12	Reconciliation by construction
3	Capability architecture	13	Multi-tenancy & isolation
4	Service decomposition	14	Identity & access
5	Data architecture	15	Data protection & retention
6	Ingestion & rail adapters	16	Reliability & scale
7	Detection runtime	17	Observability & metering
8	Inline decisioning — Lane A	18	Deployment architecture
—	incl. the UPI / IMPS API contract	19	Decision record
9	Backfill, replay & simulation	20	Known gaps
10	Case lifecycle engine	21	AI/ML roadmap — architecture

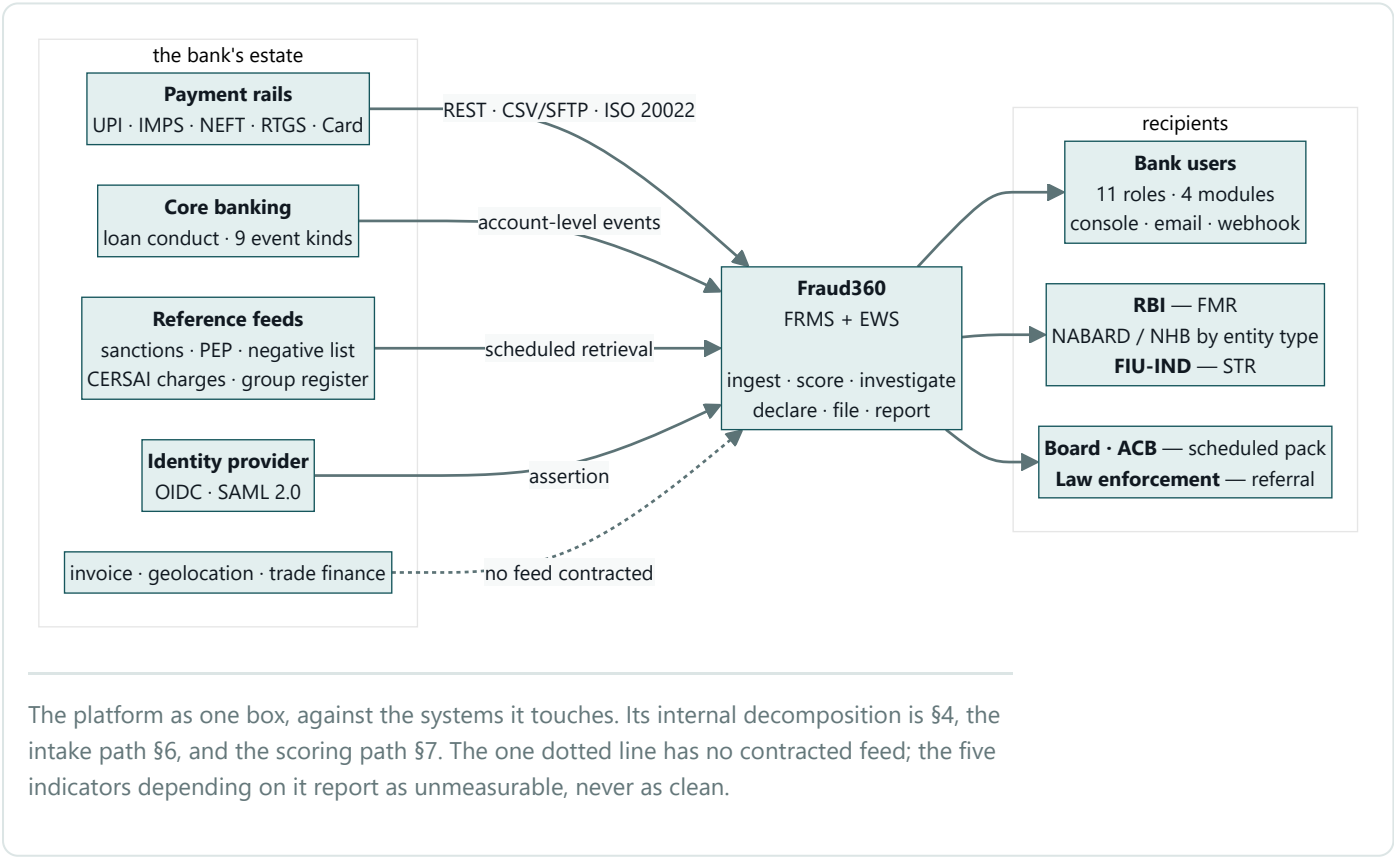
1. Design principles

Five constraints that decide most arguments before they start. Where a later section makes a surprising choice, it is usually one of these.

#	PRINCIPLE	CONSEQUENCE
P1	One definition per figure. Every metric is defined once and read by every surface	Dashboards cannot disagree. Rules out per-screen SQL and rules out eventually-consistent read models for reported metrics
P2	The control plane governs behaviour. Detection reads the tenant's configuration, never a constant	Retuning a band in the console changes what fires, with no deploy. A badly-tuned band genuinely misses fraud
P3	Fail loudly, never degrade quietly. A component that cannot do its job correctly refuses rather than approximating	Unsafe configurations refuse to boot; an unreachable catalogue halts scoring rather than inventing thresholds
P4	Procedural guarantees live in code, not policy. Natural justice, dual control and evidence requirements are enforced	The lifecycle is a state machine with guards, not a status column
P5	Absence is reported, not filled in. An indicator with no data feed stays silent and says why	The dormant register is meaningful. Nothing is approximated to make a screen look busy

2. System context

What the platform talks to, and where the trust boundaries fall.



Interfaces in detail

Every line above, with what actually crosses it. Rails normalise to one canonical transaction at the edge, so no rule ever learns what a UPI collect request looks like.

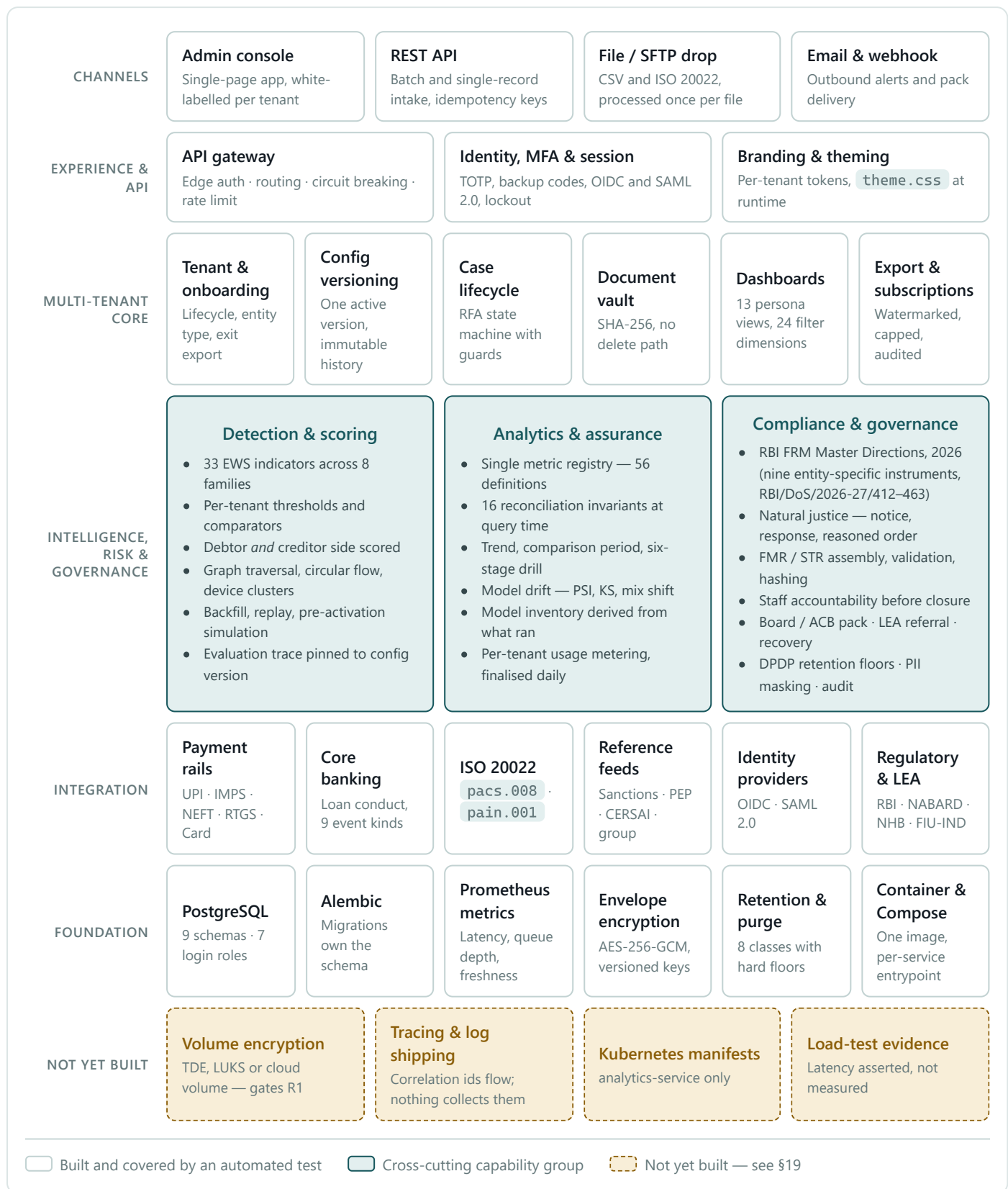
INTERFACE	DIR	TRANSPORT / FORMAT	CARRIES	CADENCE
Payment rails UPI · IMPS · NEFT · RTGS · Card	in	REST batch (JSON), CSV over SFTP, or ISO 20022 <code>pacs.008</code> / <code>pain.001</code>	Debtor and creditor account, amount in paise, timestamp with offset, rail, channel, device, counterparty. Five adapters, one canonical shape out	Near-real-time or batched file
Core banking loan conduct	in	REST, canonical event envelope	Seven kinds: loan disbursement, repayment, utilisation, sale proceeds, cash withdrawal, account status, RM observation — with funding source where known	Event-driven or daily
Reference feeds	in	HTTPS pull, CSV / JSON / line-delimited, credential sealed at rest	Five list kinds: sanctions, PEP, negative list, CERSAI charges, group register. Versioned; a materially shrunken feed is refused with the previous version left active	Scheduled per source
Identity provider	in	OIDC, or SAML 2.0 POST binding	Authenticated subject and attributes, read from the signed element. Assertion IDs recorded and replays refused	Per sign-in
Invoice · geolocation · trade finance	in	—	<i>No feed contracted.</i> SME-03, CHN-02 and TBM-01/02/03 report unmeasurable with the reason	—
Bank users	out	Console (HTTPS), email, webhook	Dashboards, case work, alerts. 11 roles across 4 modules; compliance-critical notifications ignore per-user mutes	Interactive; alerts on event
RBI — and NABARD or NHB by entity type	out	Generated artefact; format binding <code>not_bound</code> (\$10)	FMR return, assembled from the case, its evidence and the reasoned order; validated, hashed and retained	Per declaration, against the policy clock
FIU-IND	out	Generated artefact; format binding <code>not_bound</code>	STR	Per case
Board · ACB	out	Rendered pack, delivery recorded per recipient	Governance pack built from the same metric definitions the board sees on screen	Scheduled
Law enforcement	out	Referral record on the case	Referral and any recovery entries against it	Per case

WHY THE RAIL ADAPTERS SIT AT THE EDGE AND NOT IN THE RULES

A UPI collect request, an IMPS P2A transfer and a card authorisation describe the same event with different field names, different amount units and different notions of who initiated it. If that variation reaches the rule engine, every indicator has to know about every rail, and adding a sixth rail means editing thirty-three indicators. Normalising once at intake means a rail is an adapter, and the detection code has never heard of UPI.

3. Capability architecture

The same platform as a capability stack rather than a flow — what each layer provides, and which capabilities are not yet built.



Two things this view makes visible that the flow diagrams do not. The **intelligence band spans the whole width** because detection, assurance and compliance are not a layer traffic passes through — they read from and write to every layer above and below them, which is also why they are hard to extract into a separate service (§18, AD-09). And the **bottom band is part of the architecture**: a capability diagram that shows only what exists is a sales diagram, not a design one.

4. Service decomposition

Seven independently deployable services. The split follows data ownership, not code convenience.

SERVICE	PORT	OWNS	RESPONSIBILITY
gateway	8080	— (stateless)	Single edge. Verifies the token, routes to the owning service, applies rate limiting and circuit breaking, serves the console
tenant-service	8081	tenant	Tenant register, users — invite/update/remove now open to a tenant's own <code>can_admin_tenant</code> -capable role (BR-108), not only platform staff — the role catalogue — materialised per tenant at onboarding (BR-113), with create/edit/delete and a maker-checker gate on any privilege-raising change — authentication, MFA enrolment and challenge, sessions, lockout, OIDC and SAML sign-on, machine credentials for bank systems, the platform audit trail (tenant-scoped for the six roles granted it, unscoped for platform staff) with CSV export
branding-service	8082	branding	Per-tenant theme, logo, palette, custom domain
config-service	8083	config	EWS catalogue, FRM policy, versioning and activation. Publishes cache invalidation
ingestion-service	8085	ingestion	Rail adapters, file and ISO 20022 intake, CBS loan-conduct events, validation, deduplication, quarantine, the inbound queue and its claim/ack protocol
analytics-service	8084	analytics, cases	Metric registry, reconciliation, dashboards, detection runtime, RFA lifecycle, staff accountability, LEA referral and recovery, regulatory filings, board packs, reference feeds, usage metering, documents, saved views, the exit fraud-record export
decision-service	8086	decision	Lane A. Inline go/no-go inside the payment window: per-rail budget and policy, account counters, the inline-eligible rule subset, shadow-mode evaluation and the decision log. Its own service because it is the only one in the payment path, and its availability class differs from everything else here
notification-service	8087	notify	Channel configuration, per-user preferences, notification records and delivery attempts

analytics-service has grown considerably since v1.0 and is now the obvious candidate for a split. That is discussed in §18 (AD-09) rather than treated as an oversight: the `cases` schema boundary was carved for exactly this, and the reason it has not happened is a stated trade, not inattention.

WHY DETECTION LIVES INSIDE ANALYTICS-SERVICE

Detection writes the facts that analytics reads, and case management mutates them. Splitting them today would mean one service writing another's tables — the shared-database problem, made worse. The `cases` schema is already separated so the eventual split is a deployment change rather than an excavation, but it is blocked on a real trade: an event-synced read model would break the exact reconciliation P1 depends on. That decision is open (BRD OD-02).

HOW A CUSTOM ROLE REACHES FOUR SERVICES WITH NO GRANT ON TENANT-SERVICE'S SCHEMA

`cp_common.dynamic_roles` is a shared package module — written once, imported by analytics-service, config-service, decision-service and notification-service — rather than each service growing its own copy of the same fetch-and-cache logic. Every one of the ten fixed roles resolves against the in-process `rbac.ROLES` dict, unchanged, with zero network calls. Only a role name outside that set triggers a fetch to tenant-service's `GET /tenants/internal/{tenant_id}/roles`, cached against the "roles" namespace on the same generation-token mechanism (`cp_common.cache`) config-service already bumps for rule and policy changes — so a role edit is live everywhere the moment tenant-service commits it, not after a TTL. This is a deliberately different trade than decision-service's own `catalogue_for()` (Lane A, below): that path may never block, so it serves a stale copy while refreshing in the background; this one is not on the payment path and a miss is rare (a custom role at all is rare), so it blocks briefly on a miss instead, trading a bounded, one-time latency for never honouring a revoked grant even momentarily. `ingestion-service` needed no change — BR-213 machine credentials carry scopes, not human roles, so it has no role check to extend.

Three capability flags travel through this path, not two: `can_admin_tenant`, `can_reveal_pii`, and — since phase 2d — `can_activate_config`, the `risk_manager`-equivalent grant BR-715's maker-checker runs on. config-service's `maker_checker.ensure_eligible()` used to check a hardcoded name tuple (`tenant_admin`, `platform_admin`, `risk_manager`) because there was no flag narrower than `can_admin_tenant` to check dynamically; now it checks one capability, and the three fixed roles that pass it do so through exactly the same `rbac.ROLES` data every other check in this section reads, not a second, separately-maintained list.

CONFIG-SERVICE: THE RULE CATALOGUE IS NO LONGER IDENTICAL FOR EVERY TENANT (BR-111)

Onboarding used to seed the same EWS catalogue regardless of entity type — only the FRM policy body was entity-type aware. `ENTITY_TYPES` now carries a `can_extend_credit` flag per entity type, checked against each of the nine RBI 2026 Directions' own Chapter III text rather than assumed, and `rule_configs()` accepts it to drop the 20 rules whose premise is a lending relationship (loan-account misuse, borrowal-account conduct, collateral, title disputes, related-party exposure, qualitative credit signals, loan origination — BR-214) for an entity type that cannot extend credit — today, only Payments Bank. This is capability-gated seeding, not a per-tenant override: it happens once, at onboarding, in `default_configs_for()`, and what gets seeded is what `internal_active_rules` serves to decision-service and analytics-service — an excluded rule is absent from the tenant's configuration, not merely deactivated. Trade- finance rules (TBM) are deliberately not gated the same way: none of the nine Directions tie Authorised Dealer / forex authorisation to entity type, so that would need a new per-tenant signal the platform does not capture yet, not an entity-type default. See FSD §"Detection scope" for the functional detail and BRD BR-111 for the traceable requirement.

NOT YET BUILT: CENTRAL FRAUD REGISTRY REPORTING (BR-509)

Each of the nine 2026 Directions carries its own Chapter VI sub-section on reporting to RBI's Central Fraud Registry (CFR), distinct from the FMR return BR-504/505 already covers. Nothing in this codebase names CFR today. The expected shape is an extension of the same filing-artefact-to-prescribed-format binding BR-504/505 already tracks as `not_bound` pending **OD-04** — not a new cross-tenant data-sharing mechanism. That distinction matters: the tenant-isolation boundary this section treats as inviolable elsewhere (an unscoped key is a cross-tenant leak waiting to happen) is not to be weakened to build this — each tenant reports its own incidents to RBI's registry, the same way it already reports FMR/STR, not into a database shared across tenants. Not built; timing not yet committed (BRD BR-509).

5. **Data architecture**

Nine schemas in one PostgreSQL instance, each owned by one service and reachable only by that service's database role.

SCHEMA	OWNER	PRINCIPAL TABLES	NOTES
tenant	tenant-service	tenants, tenant_users, platform_users, user_sessions, mfa_backup_codes, identity_providers, sso_login_states, saml_assertions_seen, service_credentials	The only schema holding institution and user records. Sealed columns: MFA seeds, IdP client secrets, machine-credential certificate thumbprints
branding	branding-service	branding	—
config	config-service	tenant_configs	Versioned rule and policy bodies; one active per name

SCHEMA	OWNER	PRINCIPAL TABLES	NOTES
ingestion	ingestion-service	raw_transaction, ingest_files, quarantined_rows, cbs_events	Append-only inbound record with the original payload; rejected rows quarantined rather than discarded
analytics	analytics-service	fact_transaction, fact_alert, fact_case, regulatory_filings, board_packs, board_pack_distributions, reference_lists, reference_entries, reference_sources, usage_daily, saved_views, report_subscriptions	Derived facts, the analytical surface, and the artefacts that leave the building
cases	analytics-service	case_transitions, case_documents, staff_accountability, accountability_findings, lea_referrals, recovery_entries	Carved out to mark the bounded context ahead of any split
decision	decision-service	account_counters, decision_log	Lane A. Deliberately tiny and deliberately separate: everything on the payment path is a single-key lookup or arithmetic, and a schema of its own is what keeps a reporting query off that path. Its availability class differs from the rest — see §13
notify	notification-service	channel_config, preferences, notifications, deliveries	The attempt is recorded separately from the notification, so a failed delivery is visible rather than inferred
platform	shared infrastructure	audit_logs, config_generation, idempotency_keys	Deliberately small. The role a message bus or Redis would play

Ownership is enforced by PostgreSQL, not convention. Each service connects as its own login role holding **USAGE** on its own schemas plus **platform**, with a matching **search_path**, so an unqualified reach for a neighbour's table fails as *relation does not exist*. Migrations run as the owning role; no service can perform DDL at runtime.

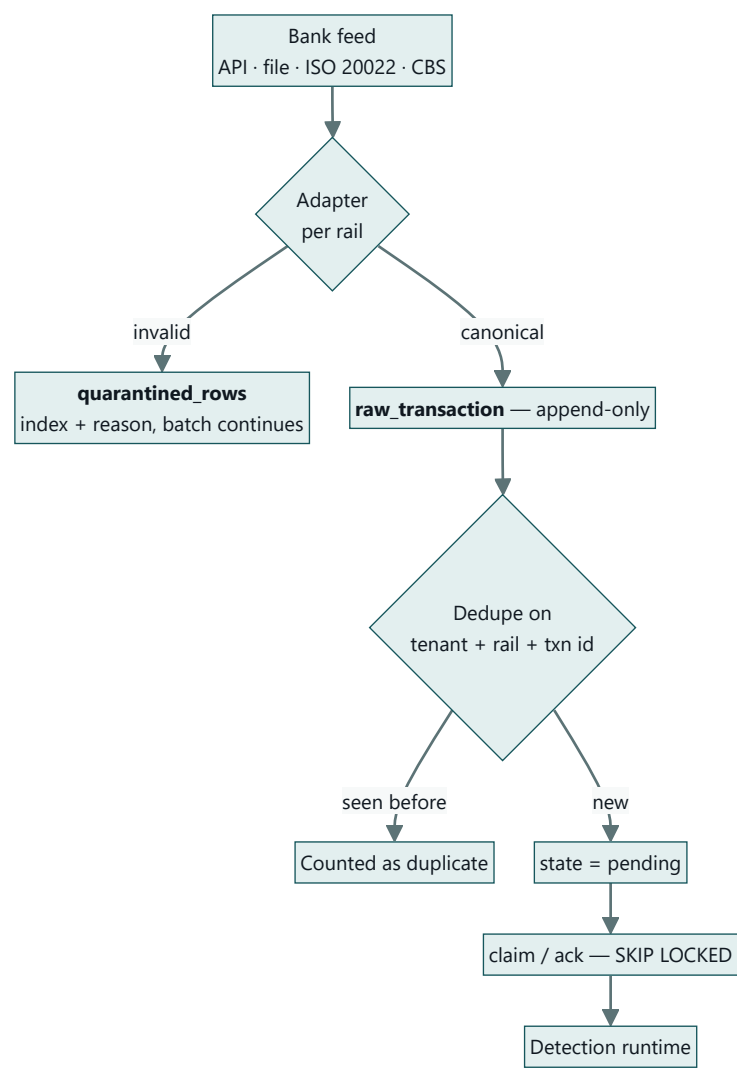
ROLE	REACHABLE SCHEMAS
svc_tenant	tenant, platform
svc_branding	branding, platform
svc_config	config, platform
svc_ingestion	ingestion, platform
svc_notify	notify, platform
svc_analytics	analytics, cases, ingestion, platform
svc_decision	decision, platform

svc_analytics holding read access to **ingestion** is the one deliberate cross-service grant: it is the projection seam, and precisely what a message bus would replace. It is documented rather than incidental, and analytics still cannot reach the tenant register.

`svc_decision` has *no* cross-grant at all. Lane A answers from its own counters and nothing else; it cannot read a transaction, a case or a tenant. That is what lets the inline path hold a 50–150 ms budget without a defensive timeout on a query that should never have been reachable.

6. Ingestion and rail adapters

One canonical transaction shape, produced at the edge, so no rule ever learns what a UPI collect request looks like.



A malformed row is reported with its index and reason while the rest of the batch proceeds. Rejecting five thousand good rows over one bad timestamp is how a bank ends up building its own retry queue and sending everything twice.

DESIGN DECISIONS THAT CARRY WEIGHT

DECISION	RATIONALE
Amounts parsed via <code>Decimal</code> to integer paise; sub-paise refused	<code>float("1234.55") * 100</code> is 123454.999999999999. Across a day's volume that is a reconciliation failing for no findable reason

DECISION	RATIONALE
Timestamps without an offset are rejected	IST is +05:30. Guessing wrong places a transaction five and a half hours from where it happened and silently breaks every time-window rule
The original payload is retained beside the canonical form	"What did you receive, and when" is a different question from "what did you conclude", and cannot be answered from a table detection also rewrites
Workers claim work over an internal API, not by reading the table	Ownership stays with ingestion-service; substituting a real queue later changes two endpoints and nothing else
<code>FOR UPDATE SKIP LOCKED</code> on claim	Several detection workers take different slices instead of queueing; a crashed worker's claim ages back into the queue

Intake channels

Four ways in, one canonical output. A bank that cannot open a firewall for an API is a normal customer, not an exception.

CHANNEL	SHAPE	NOTES
REST batch	JSON, canonical or per-rail	The synchronous path. Per-row rejection with index and reason
File / SFTP drop	CSV	Each file is registered, checksummed and processed once. Re-dropping the same file is a no-op, not a double count
ISO 20022	<code>pacs.008</code> , <code>pain.001</code>	Parsed by local element name, so a bank's own namespace prefix does not break it. <code>GrpHdr/NbOfTxS</code> and <code>CtrlSum</code> are verified against what was actually parsed, and a mismatch fails the <i>whole</i> file
CBS loan-conduct events	9 event kinds	Account-level rather than payment-level: sanction, disbursal, repayment, classification change, security, BG/LC and end-use events. Feeds the BEH and CBS indicator families

A CONTROL TOTAL THAT ONLY FAILS PART OF A FILE IS NOT A CONTROL TOTAL

ISO 20022 carries its own count and sum in the group header. The obvious implementation rejects the rows that disagree and keeps the rest — which is precisely wrong. If the declared sum does not match the parsed sum, the disagreement is about what the file *contains*; there is no basis for deciding which rows are the good ones. The file is refused whole and the bank is told the declared and computed figures. Partial acceptance under a failed control total is how two systems end up holding different versions of the same day.

Rows that fail validation are written to `quarantined_rows` with their reason rather than dropped. "We sent you 40,000 rows and you booked 39,997" is a question the platform has to be able to answer with the three rows, not with a count.

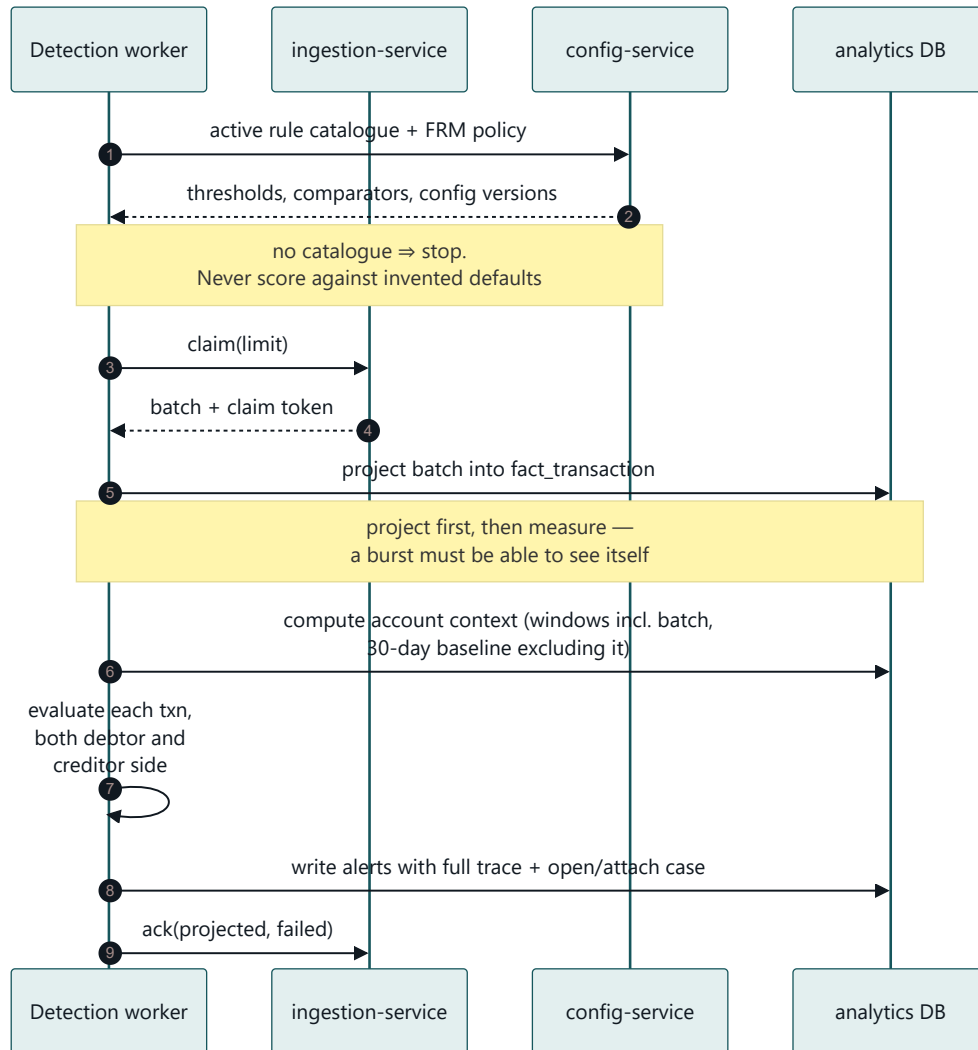
A FIFTH CHANNEL NOT YET BUILT: LOAN-ORIGINATION (LOS) SIGNALS (BR-214)

All four channels above are post-disbursement — even CBS loan-conduct events start at sanction, not at application. ~92% of FY25 reported bank fraud value is loan/advances fraud (RBI Annual Report FY25), and a falsified application or a straw borrower is invisible to this platform until money has already moved. A fifth channel reading the underwriting/LOS stage would extend the same canonical-shape-at-the-edge pattern this section already uses, not require a new architecture - but LOS systems vary far more across NBFCs and co-operative banks than core banking does, so the adapter surface is larger than the four above. Not built; timing not yet committed (BRD BR-214).

7. Detection runtime

The catalogue owns the threshold; the engine owns the measurement. Keeping

those apart is what makes P2 true.



Ordering is load-bearing. Measuring before projecting made every same-file burst invisible: seven structuring transfers each looked lonely because their accomplices had not been written yet.

Feature computation

Observations are computed per account, once per batch, rather than per transaction — a handful of aggregate queries instead of one query per row. Both sides of every payment are scored: a mule collection account may never pay anybody, so a debtor-only view never assesses the hub of a fan-in, which is the single account an investigation is about.

Thirty-three indicators are declared. Which of them can actually be measured depends on which feeds a given tenant has connected, so the split below is by *dependency*, not by whether code exists.

FAMILY	FROM TRANSACTIONS ALONE	NEEDS A CONNECTED FEED	NO FEED CONTRACTED
Velocity	VEL-01, VEL-02, VEL-03	—	—
Structuring	SME-01, SME-02	—	SME-03 (invoice)
Behavioural	BEH-01	BEH-02, BEH-03 (CBS)	—
Loan conduct	—	CBS-01, CBS-02, CBS-03, CBS-04, CBS-05, CBS-06, CBS-07, CBS-08, CBS-09, CBS-10 (CBS)	—
Layering / mule	LAY-01, LAY-02, LAY-03, LAY-04	—	—
Counterparty	CPT-01	CPT-02 (sanctions), CPT-03 (CERSAI), CPT-04 (title dispute)	—
Channel / device	CHN-01, CHN-03	—	CHN-02 (geolocation)
Trade-based	—	—	TBM-01, TBM-02, TBM-03
Total	13	15	5

LAY-03 (multi-hop layering) moved from the third column to the first: the graph traversal is now computed in the database rather than requiring an external graph store. The fifteen middle-column indicators became measurable when CBS event intake and scheduled reference retrieval landed — but only for a tenant that has actually connected those feeds, which is the distinction the next note is about.

AN UNLOADED LIST IS NOT A CLEAN SCREEN

CPT-02 screens counterparties against a sanctions list. If no list has been loaded, the tempting implementation returns "no match" — and a bank reads that as *screened, clean*. It reports the indicator as **unmeasurable, with the reason**, and the case surfaces say so. The same rule governs every dependent indicator: a proportion whose denominator is absent is omitted, never returned as zero. Loading an empty list is refused outright, and a feed that arrives materially smaller than the version it would replace is refused with the previous version left active — a truncated sanctions file is indistinguishable from a clean one at the moment it is applied, and only distinguishable afterwards by the frauds that were not flagged.

COMPARATOR DIRECTION IS PART OF THE CONFIGURATION

Most indicators fire at or above their threshold. "Beneficiary added within 24 hours" cannot be expressed that way — a two-hour-old payee is more suspicious than a twenty-hour-old one. Each rule therefore carries an explicit comparator. Without it the two age-based indicators compared the wrong way round and never fired at all: a silently dormant fraud indicator, which is exactly the finding an inspection looks for.

8. Inline decisioning — Lane A

A go/no-go answer inside the payment window, without pretending the near-real-time lane can be made to fit in one.

The detection runtime above drains a queue. It sees a transaction after it has been ingested, scores it against the full catalogue with complete account context, and raises an alert. That is the right design for an early-warning system and the wrong design for a UPI authorisation, which must be answered in a few hundred milliseconds end to end. Stretching one lane to cover both would have meant either a detection engine that skips work it should do, or a payment that waits for work it should not.

So there are two lanes, and the split is explicit:

	LANE A — INLINE	LANE B — NEAR-REAL-TIME
Question	May this payment proceed?	Is this account behaving like fraud?
Answers	Binary on UPI and IMPS — approve or decline. Card adds a 3-D Secure challenge; NEFT and RTGS can hold	Scored alerts, grouped into cases
Budget	50–150 ms by rail, set per tenant	Seconds; measured at p99 2.65 s per worker at 200 tps
Context	Pre-computed counters and what the channel supplies	Full account, graph and reference context
Rules	9 of the 33 catalogue rules are inline-eligible	All 33
Reads	Single-key lookups only; no aggregate, no cross-service call	Whatever the typology needs, including multi-hop traversal
On failure	Reports which rules were skipped; the outcome is <code>budget_exceeded</code> , never a clean pass	Retries; the backlog is visible
Availability class	In the payment path — its own service, own schema, own role, own scaling	Can absorb a restart behind the queue

WHICH RULES CAN GO INLINE IS A PROPERTY OF THE RULE, NOT A PREFERENCE

A rule is inline-eligible only if it can be answered from precomputed per-account counters or from what the channel already sends. The other 19 are listed with the reason each is excluded — a graph traversal whose cycle has not closed, a reference-list join, a CBS event posted after settlement. That register is code (`inline_rules.py`), keyed by rule id, so a new rule has to declare which lane it belongs to before it can be catalogued.

Two exclusions are about correctness rather than latency. **CPT-01** and **VEL-02** measure beneficiary age with an `lte` comparator, and the first-seen-pair set they need is not an account counter. Scored from a context without it, an unknown payee reads as age zero — and the rule fires on every high-value payment. A control that declines everything is as broken as one that declines nothing, so they are suppressed explicitly and reported unmeasured.

ONE DEFINITION PER FIGURE, ACROSS BOTH LANES

Lane A and Lane B share `cp_common.observations.observe()` . They do not each have their own idea of what "velocity" means. If the two lanes could drift apart on a definition, a tenant would eventually see an inline decline and a Lane B dashboard that disagreed about the same account, and there would be no way to say which was right.

AN ACTION THE RAIL CANNOT EXECUTE IS WORSE THAN NO ACTION

The action set is a property of the payment scheme, not a tenant preference, and it is checked when the policy is configured rather than when the payment arrives. On UPI the PIN is captured at the remitter PSP before the request reaches the bank, so the customer has already authenticated and the NPCI response set is approve or decline — there is no step-up code to return. IMPS is the same. Card is the exception: 3-D Secure makes a challenge a first-class protocol outcome. `hold` belongs only to NEFT and RTGS, which settle in a window wide enough for outward-remittance screening to be a real operation.

This was a live defect, not a hypothetical. The shipped starter policy offered `challenge` on UPI and made IMPS challenge-only — so that lane could never decline anything at all. Had a tenant enforced it, the decision log would have recorded a control firing, the switch would have ignored an answer it did not recognise, and the payment would have settled. Every report would have shown the control working. A rail with no declared capabilities is refused rather than assumed permissive, so a new scheme cannot inherit the most powerful set by default.

A second front door onto the same core: ISO 8583 (BR-317, Partial)

`/decide` above is JSON, reached directly at the decision-service host. Some card/EFT switches — frequently on-prem, vendor-licensed, no REST layer available without a change request per bank — have no practical way to originate that call. Asking each such vendor to build a translation layer of their own does not scale: it is N bespoke integration projects on infrastructure Fraud360 does not control, instead of one adapter built and tested once.

`iso8583_gateway` is that one adapter: a TCP listener, its own deployable for the same reason decision-service already is — a raw-socket connection has a different failure mode than an HTTP service behind the gateway, and a hung 8583 connection must not affect the JSON path or vice versa. It decodes an authorisation message (MTI 0100/0120), maps the standard data elements to a `DecideIn` body, and calls `POST /decide` exactly as any JSON integrator would — same rule catalogue, same per-rail policy, same decision log. **Two front doors, one brain**, deliberately: there is exactly one function in the platform that ever produces a decision, and this gateway is a client of it, not a second implementation of it.

ISO 8583 DE	CANONICAL FIELD
DE2 (PAN)	<code>debtor_account</code> — masked at the parse boundary (SHA-256 + last 4), never the raw PAN
DE4 (amount)	<code>amount_paise</code>

ISO 8583 DE	CANONICAL FIELD
DE7 (transmission date/time)	<code>ts</code> — MMDDhhmmss carries no year and no offset; the year is inferred against a configured local offset (no default — see below), rolling back across a Dec→Jan boundary
DE11 / DE37 (STAN / RRN)	<code>txn_ref</code>
DE41 (terminal id)	<code>device_id</code>
DE42 (card acceptor id)	<code>creditor_account</code>

WHAT IS BUILT, AND WHAT IS DELIBERATELY STILL OPEN

The codec, field mapper and connection handler are complete and unit-tested (45 tests): message round-trip including LLVAR fields and leading-zero-preserving fixed fields, the DE7 year inference including the year-boundary case, PAN masking, and every failure path — a malformed message, an unreachable `/decide`, a `/decide` response this gateway cannot represent — producing an explicit refusal (DE39 96/91) rather than a guessed approval. `iso8583_fail_open` has no default, by the same discipline `RailPolicy.fail` already applies to every inline rail: the process refuses to start rather than pick open or closed on the bank's behalf.

Not built, and not guessable: DE48/62/120–127 are private-use in the 1987 standard and every switch vendor fills them differently, so this gateway maps only the public data elements until a named pilot vendor's own spec confirms the rest — see `docs/ISO8583_LANE_A_SCOPING.md`. Whether terminating a raw PAN here brings Fraud360 into PCI-DSS cardholder-data scope is a compliance decision, not an engineering one, and is likewise still open. Not deployed for any tenant.

The contract — UPI and IMPS worked end to end

Every payload below is a verbatim capture from the running services, not an illustration. Only the bearer token is abbreviated, and the `skipped` array is truncated where noted — nothing else is edited.

1 · THE SWITCH AUTHENTICATES

Once per token lifetime, not once per payment. There is no MFA on this path and none is being bypassed: a machine credential has no interactive login and never can — see §13. The token is a bearer token scoped to `decide`; possession is additionally bound to a client certificate where the bank can offer mTLS.

REQUEST

```
POST /machine/token
Content-Type: application/json

{
  "grant_type": "client_credentials",
  "client_id": "svc_3063a96d7d0b17e1",
  "client_secret": "<shown once at creation, never retrievable>"
}
```

RESPONSE — 200

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6I..",
  "token_type": "Bearer",
  "expires_in": 1800,
  "scope": "ingest+decide",
  "tenant_id": "b3b90261-9663-46c3-b31f-e8afc4a65442"
}
```

2 · A UPI PAYMENT THAT PASSES

`signals` carries what the channel already knows at authorisation time. `risk_signals` is the one observation the lane will not derive for itself: read from a rebuilt context it would count every payment as a new device paying a new payee, because that context holds counters, not sets. The channel is the better-informed party here, so its figure is used or the rule is reported unmeasured.

REQUEST

```
POST /decide?tenant_id=b3b90261-9663-46c3-b31f-e8afc4a65442
Authorization: Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6I..
Content-Type: application/json

{
  "txn_ref": "UPI-2026-0805-000191",
  "rail": "UPI",
  "debtor_account": "AC-HLD-CALM",
  "creditor_account": "AC-BEN-77120",
  "amount_paise": 250000,           // integer paise, never a float
  "channel": "mobile",
  "device_id": "d41d8cd98f00b204",
  "signals": { "risk_signals": 0 }
}
```

RESPONSE — 200 IN 4.2 MS

```
{
  "reference": "72492b3d91e24a4b",
  "action": "allow",                // what the switch must act on
  "would_be": "allow",             // what enforcement would have returned
  "outcome": "clean",              // rules ran; none matched
  "score": 0.0,
  "severity": "low",
  "enforced": false,               // this rail is still in shadow
  "shadow": true,
  "matched": [],
  "skipped": [
    { "rule_id": "CPT-01",
      "reason": "beneficiary age needs the first-seen-pair set, which is not an
                account counter. It is an 'lte' rule, so an unknown pair would
                read as age zero and fire on every high-value payment",
      "lane": "near-real-time" },
    { "rule_id": "VEL-01",
      "reason": "not measurable for this payment from account counters",
      "lane": "inline" }
    // ... 20 more, each with its reason. Truncated for this document only.
  ],
  "took_ms": 4.16,
  "budget_ms": 80,
  "policy_version": "1.0.0"
}
```

OUTCOME: "CLEAN" IS A CLAIM, AND IT HAS TO BE EARNED

It means every inline-eligible rule ran and none matched. It is *not* what comes back when the catalogue is empty (`not_screened`), when the counter store is unreachable (`store_unavailable`), or when the budget expired part-way (`budget_exceeded`). Those are separate outcomes precisely because all four would otherwise arrive at the switch as `allow` and be indistinguishable in the record afterwards.

The `skipped` array is the other half of that claim. A rule that could not be measured is named with its reason rather than counted as a pass, so "we screened this payment" can always be qualified by "against these rules, and not these".

3 · A UPI PAYMENT THAT WOULD BE DECLINED

The same endpoint, an account whose counters describe a mule: value far above its own baseline, fifty-five distinct counterparties, near-total outflow, structuring below the CTR threshold, and a new device on a new payee.

REQUEST

```
{
  "txn_ref": "UPI-2026-0805-000192",
  "rail": "UPI",
  "debtor_account": "AC-HLD-HOT",
  "creditor_account": "AC-BEN-99001",
  "amount_paise": 500000,
  "channel": "mobile",
  "device_id": "9e107d9d372bb682",
  "signals": { "risk_signals": 5 }
}
```

RESPONSE — 200 IN 1.7 MS

```
{
  "reference": "77de6e3a0eba4fb6",
  "action": "allow",
  "would_be": "decline",
  "outcome": "matched",
  "score": 600.0,
  "severity": "critical",
  "enforced": false,
  "shadow": true,
  "matched": [
    { "rule_id": "CHN-01", "family": "CHN", "sub_rule_ref": ".03",
      "observed": 5.0, "threshold": 3.0, "observed_unit": "risk_signals",
      "reason": "New device + new beneficiary + high value in one session" },
    { "rule_id": "LAY-02", "family": "LAY", "sub_rule_ref": ".03",
      "observed": 55.0, "threshold": 40.0, "observed_unit": "distinct_counterparties",
      "reason": "40+ distinct counterparties in window (fan-in/fan-out hub)" },
    { "rule_id": "LAY-01", "family": "LAY", "sub_rule_ref": ".03",
      "observed": 0.98, "threshold": 0.9, "observed_unit": "outflow_ratio",
      "reason": "Outflow >= 90% of recent inflow drained within window" },
    { "rule_id": "SME-01", "family": "SME", "sub_rule_ref": ".03",
      "observed": 7.0, "threshold": 5.0, "observed_unit": "sub_threshold_count",
      "reason": "5+ transfers just below the CTR threshold in 24h" },
    { "rule_id": "VEL-01", "family": "VEL", "sub_rule_ref": ".02",
      "observed": 50.0, "threshold": 5.0, "observed_unit": "value_vs_baseline",
      "reason": "Transaction value 5x the 30-day baseline" }
  ],
  "skipped": [ /* 20 entries, each with its reason */ ],
  "took_ms": 1.663,
  "budget_ms": 80,
  "policy_version": "1.0.0"
}
```

Every match carries **what was observed, what it was compared against, in what unit, and the band's own wording**. That is the same trace the near-real-time lane records on an alert, so a declined payment is explainable months later on the terms an investigator already knows — and `policy_version` says which configuration was in force, because the thresholds will have moved on.

4 · THE SAME CALL ON IMPS

Identical contract; only the budget differs, and it comes from the tenant's own per-rail policy. UPI is given 80 ms and IMPS 150 ms because their settlement windows differ, not because the code does.

REQUEST

```
{
  "txn_ref": "IMPS-2026-0805-004417",
  "rail": "IMPS",
  "debtor_account": "AC-HLD-HOT",
  "creditor_account": "AC-BEN-99002",
  "amount_paise": 4500000,
  "channel": "netbanking",
  "device_id": "9e107d9d372bb682",
  "signals": { "risk_signals": 4 }
}
```

RESPONSE — 200 IN 1.5 MS

```
{
  "reference": "73c5b65625cb4a7d",
  "action": "allow",
  "would_be": "decline",
  "outcome": "matched",
  "score": 600.0,
  "severity": "critical",
  "enforced": false,
  "shadow": true,
  "matched": [ /* VEL-01 observed 450.0 against a threshold of 5.0 — the amount is
               450x this account's own 30-day baseline */ ],
  "took_ms": 1.476,
  "budget_ms": 150,           // IMPS, not UPI's 80
  "policy_version": "1.0.0"
}
```

5 · A RAIL THIS LANE DOES NOT HANDLE

NEFT and RTGS settle in a window wide enough to score them with full account and network context after ingestion, which finds more than any 80 ms check can. Asking the inline lane about one is a configuration error, and it is **refused**.

RESPONSE — 409

```
{
  "error": {
    "code": "rail_not_inline",
    "message": "Rail 'NEFT' is configured for the near-real-time lane. It is scored
               with full context after ingestion, not in the payment window."
  }
}
```

WHY THIS IS A REFUSAL AND NOT AN ALLOW

Answering `allow` would be the natural-looking thing to do — the lane has no opinion, so let the payment through. It is the wrong answer, because the switch cannot tell that response apart from a payment that was screened and passed. The rail would appear on every report as covered by inline decisioning while nothing had looked at it. The same reasoning gives `409 no_rail_policy` for a rail nobody has configured at all: an unconfigured rail must never read as a screened one.

STATUS	CODE	WHEN	WHAT THE SWITCH SHOULD DO
200	—	A decision was reached	Act on <code>action</code> . Ignore <code>would_be</code> — it is for reporting
401	<code>invalid_client</code>	Unknown client id or wrong secret	Do not retry with the same credential; alert
403	<code>credential_unusable</code>	Expired or revoked, with the reason	Rotate. Expiry is deliberate, not a fault
403	<code>ip_not_allowed</code> · <code>certificate_mismatch</code>	Address off the allow-list, or the certificate does not match the binding	Treat as a security event, not a config bug
403	<code>tenant_suspended</code> · <code>tenant_offboarded</code>	The tenant's service is stopped (§13)	Stop calling. The fraud record remains exportable
403	scope refusal	An <code>ingest</code> -only credential asked for a decision	Mint a credential scoped <code>decide</code>
409	<code>rail_not_inline</code>	The rail is routed to the near-real-time lane	Do not call for this rail. It is screened, in Lane B
409	<code>no_rail_policy</code>	No inline policy exists for this rail	A configuration gap. Escalate rather than assume it is safe

WHAT THE SWITCH MUST DO WHEN WE ARE THE ONES WHO FAIL

If the decision service is unreachable or slow, the caller applies its own timeout and the rail's **fail policy** decides — `open` or `closed`, chosen per rail by the bank and never defaulted by us. The one outcome the platform will not produce is silence dressed as approval: whenever the lane cannot answer properly it says so in `outcome`, and that value is on the record whichever way the payment went.

Shadow mode

Every rail ships with `shadow = true`. The service computes the decision, records what it *would* have returned, and returns `allow`. A tenant can therefore run Lane A against real traffic for as long as it likes and measure the false-positive cost of enforcement before anyone's payment is declined. Turning enforcement on is a per-rail configuration change with a board-approval trail, not a deployment.

A rail with no inline policy is **refused**, not allowed. Answering "allow" for an unconfigured rail would make it look screened; a 409 makes the omission visible. The same applies to a rail explicitly configured for Lane B — asking Lane A about it is a configuration error, and it is reported as one.

The decision log is written *after* the answer is formed, and a failed write is swallowed deliberately. Losing a log line is bad; making a payment wait on our disk is worse. That is the one place in the platform where absence is not reported to the caller, and it is a considered trade rather than an oversight.

9. Backfill, replay and simulation

Retuning a threshold is worthless if nobody can see what it would have done. One code path answers all three questions.

MODE	READS	WRITES	USED FOR
Backfill	History already in <code>fact_transaction</code>	Alerts and cases	Scoring a period that predates a newly activated rule
Replay	Same	Alerts and cases	Rescoring after the catalogue changes
Simulate	Same	Nothing	"What would this threshold have fired?" before activation

Simulation is the same engine with the write suppressed and the candidate threshold substituted for the active one, so the answer it gives is the answer activation would produce. A separate estimator would be a second implementation of the rule semantics, and the first time the two disagreed nobody would know which was right.

A WINDOW'S WORTH OF CONTEXT CANNOT MEASURE A WINDOW

Backfill originally built one account context at the *end* of the requested range and scored every transaction against it. For a single-day run that is correct and it passed. Over a multi-day range it is not: a 24-hour aggregate computed at the range end describes the last day, so LAY-01, LAY-02 and VEL-03 — every rule with a 24-hour window — could not fire on any earlier day. Worse, because the counts were computed from the wrong baseline, **threshold overrides had no observable effect at all**: the simulation returned nearly the same answer whatever number was supplied, which is the most misleading possible behaviour for a feature whose entire purpose is to show the difference a threshold makes.

The run is now sliced per day and a fresh context is built for each. Verified by sweeping the threshold: LAY-02 returns 236 hits at the previous behaviour, 2,185 at a threshold of 2, and 0 at 10^6 — and a simulate run still writes zero alerts. A monotonic response to the parameter is the property that was missing, and it is now asserted by test.

10. Case lifecycle engine

The RFA lifecycle as an enforced state machine. Guards, not guidance.

GUARD	BEHAVIOUR	BASIS
Role	Each transition names the roles that may initiate it	Segregation of duties
Reason	Consequential transitions require written justification, stored on the case	Auditability

GUARD	BEHAVIOUR	BASIS
Evidence	Fraud cannot be declared without a reasoned order attached	<i>SBI v. Rajesh Agarwal</i>
Elapsed time	The borrower's response window cannot be cut short by anyone	<i>SBI v. Rajesh Agarwal</i>
Dual control	Fraud declaration records a proposal; a different user in an approver role confirms it	Maker-checker
Lateness	An action outside its policy window proceeds but is permanently marked as a breach	Blocking pushes the process off-system; hiding it deceives an inspection
Accountability	A declared fraud cannot be <i>closed</i> until the staff accountability examination is concluded	RBI expects the examination; closing without it loses the only moment anyone will do it

WHERE THE ACCOUNTABILITY GUARD IS DELIBERATELY ABSENT

The guard sits on `close_case` and **not** on `file_fmr`. RBI is explicit that reporting must not wait for the staff accountability exercise — putting the guard one transition earlier would use a compliance control to delay a regulatory filing, which is the opposite of its purpose. The eleven transitions are declared in one table with their guards as data; this one carries a comment saying why the flag is off, because a future reader will otherwise assume it was forgotten.

Every attempt is written to an append-only transition log — *including refusals*. A pattern of someone repeatedly trying to shorten a natural-justice window is exactly what must not be invisible. Case rows are locked with `SELECT ... FOR UPDATE` for the duration of a transition, so two replicas cannot both pass the same guard; without it, two concurrent requests demonstrably both applied.

Blocked actions are returned to the console **with the reason they are blocked** and rendered disabled rather than hidden. A missing button reads as a broken page; a disabled one saying *the response window is open until 20 Aug* teaches the process.

11. Regulatory artefacts

Three things leave the building: the FMR return, the STR, and the board or ACB pack. All three are assembled from the metric registry, so none can disagree with the console.

Assembly, validation and refusal

A return is built from the case, its evidence, the reasoned order and the accountability findings. It is validated against a declared field schema, hashed, and retained with the hash so that "what did we file, and when" is answerable years later without trusting the current state of the case.

A return with gaps is refused rather than generated with blanks. A half-filled statutory form that looks complete is worse than no form: someone signs it. The refusal names the missing fields, which is also the fastest route to a complete one.

THE FORMAT BINDING IS DECLARED, AND IT IS DECLARED `NOT_BOUND`

The platform assembles, validates, hashes and retains the return, and a named officer can submit from the pack. What it does *not* claim is that the file is in RBI's or FIU-IND's prescribed format — because the prescribed schemas have not been supplied. Rather than quietly emitting a plausible-looking CSV, the format binding is an explicit field whose value is `not_bound`, and it is visible on the artefact. This is the single blocker on release gate R2 (BRD BR-504/505, open decision OD-04). It needs the published schemas, not engineering.

The STR path itself is now verified end-to-end, not merely designed: a defect left the Principal Officer field — mandatory on every STR under PMLA Rule 7 — with no source anywhere in the platform, so generation was refused for every case regardless of format binding. The field now reads from the tenant's board-attested FRM policy (rides the same `board_approved` / attestation mechanism as every other governance fact in that policy, rather than a separate approval path invented for it) and has been confirmed against a live case through the real API. A tenant admin can now set that field themselves: `ConfigCreate.kind` was refusing `policy` at the request schema, blocking the console and the API both; that is fixed, and creating a new policy version still requires the same board attestation to activate as every other policy fact.

Board and ACB pack

Scheduled, versioned, and rendered from the same metric definitions the board sees on screen — the pack cannot become a separately maintained set of numbers. Distribution is recorded per recipient, so "who received the September pack" is a query rather than an email search.

Referral and recovery

Law-enforcement referrals and recovery entries hang off the case. Recovery re-derives the case's recovered total under a row lock rather than incrementing a counter, and a recovery that would exceed the loss is refused. `write_off` is deliberately absent from the recovery vocabulary: writing off a fraud loss is an accounting decision, and recording it as *recovery* would flatter the one figure a board is most likely to quote.

12. Reconciliation by construction

How P1 is actually achieved, and how it is proven at query time.

A central registry defines each metric once — its SQL expression, its base entity, its unit and its filter semantics. Dashboards do not write SQL; they name metrics. Two surfaces showing "fraud value" are therefore reading the same definition, and cannot drift apart.

Sixteen invariants run at query time and are displayed beside the figures. Integrity checks run tenant-wide; consistency checks run under whatever filters the user has applied, so the assurance matches the view being looked at.

AN INVARIANT THAT WAS WRONG FOR MONTHS

R-03 asserts a case's value equals the value of its underlying transactions. It joined case → alert → transaction, counting a transaction once *per alert*. That passed only because the synthetic generator emitted exactly one alert per transaction. Live detection fires every matching rule, so a ₹9.87 lakh payment tripping five rules appeared as ₹49.3 lakh of evidence and seven cases failed immediately. The engine was right; the check was wrong. It now sums distinct transactions. The same flaw existed in the per-case evidence panel.

Worth recording as a design lesson: an invariant validated only against generated data is validated against that generator's assumptions, not against reality.

13. Multi-tenancy and isolation

Four independent layers, so no single mistake exposes one bank's data to another.

LAYER	MECHANISM
Token	The tenant is a claim in the bearer token, not a request parameter
Route	Every endpoint resolves tenant scope before doing work; a mismatch is 403 regardless of role
Query	Every query predicates on tenant id; shared artefacts such as saved views additionally filter by owner and audience
Database	Per-service roles with schema grants; no service can reach data outside its domain

Configuration is tenant-scoped end to end. Entity type and, for UCBs, supervisory tier select the governing Direction, the reporting lines (RBI, NABARD, NHB, FIU-IND) and the default policy windows — which the tenant then reviews and owns. A Tier-1 co-operative bank and a large commercial bank run the same code against genuinely different rules.

14. Identity and access

v1.0 recorded authentication as the platform's largest open risk. This section replaces that one; what remains open is now infrastructure, not application design.

CONTROL	DESIGN
Second factor	TOTP, enrolled with a QR code and confirmed by a live code before it is enabled — an enrolment that is never verified locks the user out. Required by role, per tenant policy
Recovery	Single-use backup codes, shown exactly once and stored hashed. The screen requires an explicit acknowledgement before it can be left
Federated sign-on	OIDC and SAML 2.0. Discovery by organisation short name, so a user never has to know which protocol their bank uses
Sessions	Access tokens live 30 minutes and are deliberately unrevocable; revocation acts on the session. Absolute lifetime 7 days, idle timeout 4 hours — a session unused over a long weekend ends even though its absolute window has not run out

CONTROL	DESIGN
Lockout	5 consecutive failures, 15 minutes. Long enough to defeat credential stuffing, short enough that locking out a named colleague for a day is not a cheap attack
Authorisation	11 roles across 4 modules; every transition names the roles that may initiate it
Machine callers	A separate credential class, not a user with MFA switched off. OAuth2 client-credentials; no interactive login path and none obtainable. Scoped <code>ingest</code> , <code>decide</code> or both — never full access — with optional certificate binding, an IP allow-list and enforced expiry

SIGNATURE WRAPPING IS THE SAML FAILURE THAT MATTERS

The classic SAML break is not a forged signature — it is a valid signature over a document element that is not the one the code goes on to read. An attacker keeps the genuine signed assertion, wraps it, and adds their own unsigned assertion elsewhere in the tree; naïve verification passes, and the application then reads the attacker's element.

The implementation therefore reads the subject and attributes *from the element the signature actually covers*, rather than verifying and then re-querying the document. Exclusive canonicalisation is delegated to a maintained library rather than hand-rolled — canonicalisation bugs are silent and produce a validator that accepts documents it should not. Assertion IDs already seen are recorded and refused, so a captured assertion cannot be replayed.

WHY A PAYMENT SWITCH IS NOT A USER WITH MFA DISABLED

A switch, a CBS extract job and an ETL pipeline all have to call intake, and none of them can complete a TOTP challenge. The tempting fix is to exempt one user account. That is wrong twice: it creates a human-shaped account whose second factor is off — a switch someone eventually flips on a real person — and it attributes every ingested batch to whichever employee's password went into the switch's config.

MFA exists because secrets leak, so removing it has to be paid for rather than simply dropped. What replaces it here is **scope** (a stolen intake key cannot read a case, a customer or a dashboard), **certificate binding** (possession, where the bank can offer mTLS), an **address allow-list**, and **expiry with overlapping rotation**. Where the bank terminates TLS at its own edge and forwards the thumbprint in a header, the credential records that it proves trust in the edge rather than possession — the two must never read as equivalent in an IS review.

The standard principal dependency **refuses machine tokens by default**. Endpoints opt in. Defaulting the other way would silently expose every new endpoint until someone remembered to close it.

Tenant lifecycle as an access control

Suspension used to gate the console login and nothing else: a suspended bank's staff were locked out while its payment switch kept posting and its channel kept asking for inline decisions. That is the wrong half — the machine path carries the volume and the

billing. The lifecycle check now sits on the single dependency every machine caller passes through, so intake and decisioning are both covered, and a new machine endpoint inherits it.

Intake and decision services have no grant on the tenant register, so standing is fetched over the internal API and cached for 15 seconds — a switch at thousands of transactions a second cannot take a cross-service hop per call. The cost is stated rather than hidden: a suspension bites within that window, not instantly. If tenant-service is unreachable the last known answer is used and *marked stale*; if nothing was ever known the tenant is served, because a tenant-service outage must not become a payments outage. That is the one deliberate fail-open in the platform, and it is recorded on the result so it can be audited rather than assumed.

A suspended tenant can still export. The refusal message says so, because the bank's first question on being suspended is whether its data is gone.

Privacy of customer data

Customer identifiers are masked server-side by default — masking happens before the data leaves the service, not in the browser. Unmasking requires both a role capability and a recorded justification, and the act of viewing customer-level evidence is itself audited, not merely changes to it.

Capability is withheld from aggregate-only roles (board, CRO), from model work, and from **platform staff**. Operating the service does not require seeing a bank's customers, and an outsourcing review asks that question first.

Console output encoding

Every value interpolated into markup in the console is escaped at the point of interpolation. This was not originally true: an analyst's typed disposition reason reached the DOM unescaped, which is stored cross-site scripting in a screen shown to supervisors and, in the board module, to directors. A checker walks every markup-bearing template literal and fails the build on an unescaped interpolation, with each of the handful of exemptions carrying a written reason. Style guidance would not have held; roughly 190 call sites needed fixing.

15. Data protection and retention

Two obligations that pull in opposite directions — DPDP says delete, RBI says keep — resolved by making the floors non-negotiable.

Secrets at rest

MFA seeds, identity-provider client secrets and reference-feed credentials are sealed with AES-256-GCM under a versioned key. The record's identity is bound in as additional authenticated data, so a ciphertext lifted from one row and pasted into another fails to open rather than silently decrypting into the wrong context. Sealing is a column type rather than call sites, so a new secret column cannot be added *unsealed* by omission, and re-sealing an already-sealed value is a no-op so a re-run cannot double-wrap. With no key configured, writing a secret **raises** rather than storing plaintext.

What this does not cover is the volume itself. Full-disk or tablespace encryption protects a stolen disk or a discarded backup and is a deployment control — Postgres TDE, LUKS, or the cloud provider's volume encryption. An IS review asks for both layers; this is the outstanding half of BRD BR-712.

Retention

Eight retention classes, each with a configurable period and a **floor that configuration cannot go below**. A tenant can choose to keep records longer; it cannot choose to keep a declared fraud for six months.

CLASS	FLOOR	DEFAULT	NEVER PURGED WHILE...
Sessions and login state	7 d	90 d	—
Delivered notifications	30 d	180 d	—
Raw inbound transactions	90 d	400 d	the row is not yet projected
Alerts never linked to a case	365 d	730 d	the alert is linked to a case
Transaction facts	1,095 d	1,825 d	the transaction is evidence on a case
Closed cases that were not fraud	1,095 d	1,825 d	the case was declared fraud, or is open
Platform audit trail	1,825 d	2,920 d	—
Declared frauds and their returns	2,920 d	3,650 d	any return is unfiled, or the case is open

The purge runs dry by default, reports what each class would remove, and executes under the owning database role rather than a service role. Each class is evaluated in its own savepoint: without that, one failing predicate aborted the surrounding transaction and every later class reported zero — indistinguishable, on the screen, from having nothing to purge.

REPORTING ZERO IS A FAILURE MODE, NOT A RESULT

Two classes once reported "0 eligible" for a week because the predicate named a timestamp column that does not exist on those tables — sessions carry `issued_at` and audit rows carry `ts`. Nothing errored; the queries were simply never valid. A schema test now plans every predicate against the live database, so a column that has been renamed fails the build instead of producing a purge that quietly does nothing.

Exit — what the bank takes with it

The outsourcing directions require data return in a usable format when the relationship ends. The bundle used to contain the tenant record, users, branding, thresholds and the audit trail — all real, but none of it the part a bank cannot rebuild. The **fraud record** is: cases, the alerts underneath them, the transactions those alerts were raised on, the transition log including refusals, the accountability examinations and findings, LEA referrals, recoveries, and the filed FMR/STR returns with their payloads and content hashes. Those carry statutory retention obligations that outlive the contract and exist nowhere else.

Two deliberate omissions, both counted rather than dropped. The **full transaction history** is not in the bundle — a tenant with thirty million rows does not want them in a JSON document, and pretending otherwise produces something nobody can open; the count is stated and the bulk CSV path is named. **Document bytes** are not in the bundle either; each appears in a manifest with its SHA-256, so what is downloaded separately can be verified against what we held.

A PARTIAL BUNDLE MUST NEVER LOOK COMPLETE

The previous implementation swallowed a failed branding or configuration fetch and returned the bundle as though nothing were missing. On an exit export that is the worst place in the system for a silent gap, because the bank discovers it after the tenant is gone. Every section that fails is now recorded as `null` with the reason, and the bundle carries an explicit `complete` flag so a caller can tell a whole bundle from a partial one without inspecting every section. The flag earned itself immediately: it caught an unbound query parameter in the platform's own export on the first end-to-end run.

16. Reliability and scale

The platform is designed to run as several replicas of each service behind a load balancer. Three mechanisms make that safe.

Configuration coherence across replicas

Tenant configuration was cached per process with a time-to-live. With four replicas that meant the same dashboard, refreshed twice, could return two different sets of numbers depending on which replica served it — for a platform whose central claim is reconciliation, not staleness but wrongness.

Caching is now keyed to a **shared generation token**. Every replica reads the same counter; config-service bumps it on change; a replica whose cached copy predates the current generation discards it. The generation read is authoritative on every call — any non-zero backstop TTL simply reopens the same bug with a shorter window. A deployment configured with a process-local cache and more than one replica *refuses to start*.

Failure isolation at the edge

MECHANISM	BEHAVIOUR
Circuit breaker, per upstream	After repeated failures, calls to that upstream fail immediately instead of occupying a connection for the full timeout. One probe is admitted after the cool-down — a thundering herd re-breaks a service at the moment it recovers
Split timeouts	Connect 2s, read 25s. A dead upstream does not deserve thirty seconds; a slow analytical query does
Bounded pool	One slow upstream cannot consume every connection the gateway has — the specific way a degraded service used to take the whole console down
Retry	Reads always; writes only when the caller supplied an idempotency key

MECHANISM	BEHAVIOUR
Rate limit	Token bucket per caller. In-process, so with N replicas the effective ceiling is $N \times$ — a safety valve against runaway clients, not a billing control

Measured with analytics-service stopped: the first call failed after retries, the breaker opened, subsequent calls returned in about ten milliseconds, and the tenant and audit endpoints continued serving normally throughout.

Idempotency

A write labelled with an `Idempotency-Key` is executed once; a retry replays the stored response. Three cases are distinguished rather than collapsed: same request already finished (replay), same request still running (409 — the first attempt may yet commit), and a *different* request under the same key (422 — a client bug that must not silently discard either request). Keys are scoped to the authenticated principal; an unscoped key is a cross-tenant leak waiting to happen.

Liveness versus readiness

Liveness deliberately does not touch the database. If it did, one brief database outage would fail liveness on every replica simultaneously and the orchestrator would restart the entire fleet — turning a dependency wobble into an outage. Readiness does check dependencies, returning 503 so the load balancer removes that replica from rotation.

17. Observability and metering

Instrumentation is in place; centralised collection is not.

CAPABILITY	STATE
Correlation id propagated across every service and returned on every response, including 401 and 404	Built
Liveness, readiness, cache topology and circuit state exposed per service	Built
Immutable platform audit trail of every action, actor and outcome	Built
Prometheus text exposition per service: request counts and latencies, circuit state, queue depth and pipeline freshness	Built
Per-tenant usage metering, finalised daily	Built
Distributed tracing	Pending
Central log shipping and alerting on SLOs	Pending

The exposition format is written directly rather than pulling in a client library. It is a small, stable text format, and the dependency would have been carried into every service for the sake of a few dozen lines.

Pipeline freshness is the gauge that matters. Request latency tells you the console is responsive; it says nothing about whether detection has fallen an hour behind. A platform whose entire purpose is early warning fails most damagingly by being *silently*

late, and no request-level metric detects that.

Metering

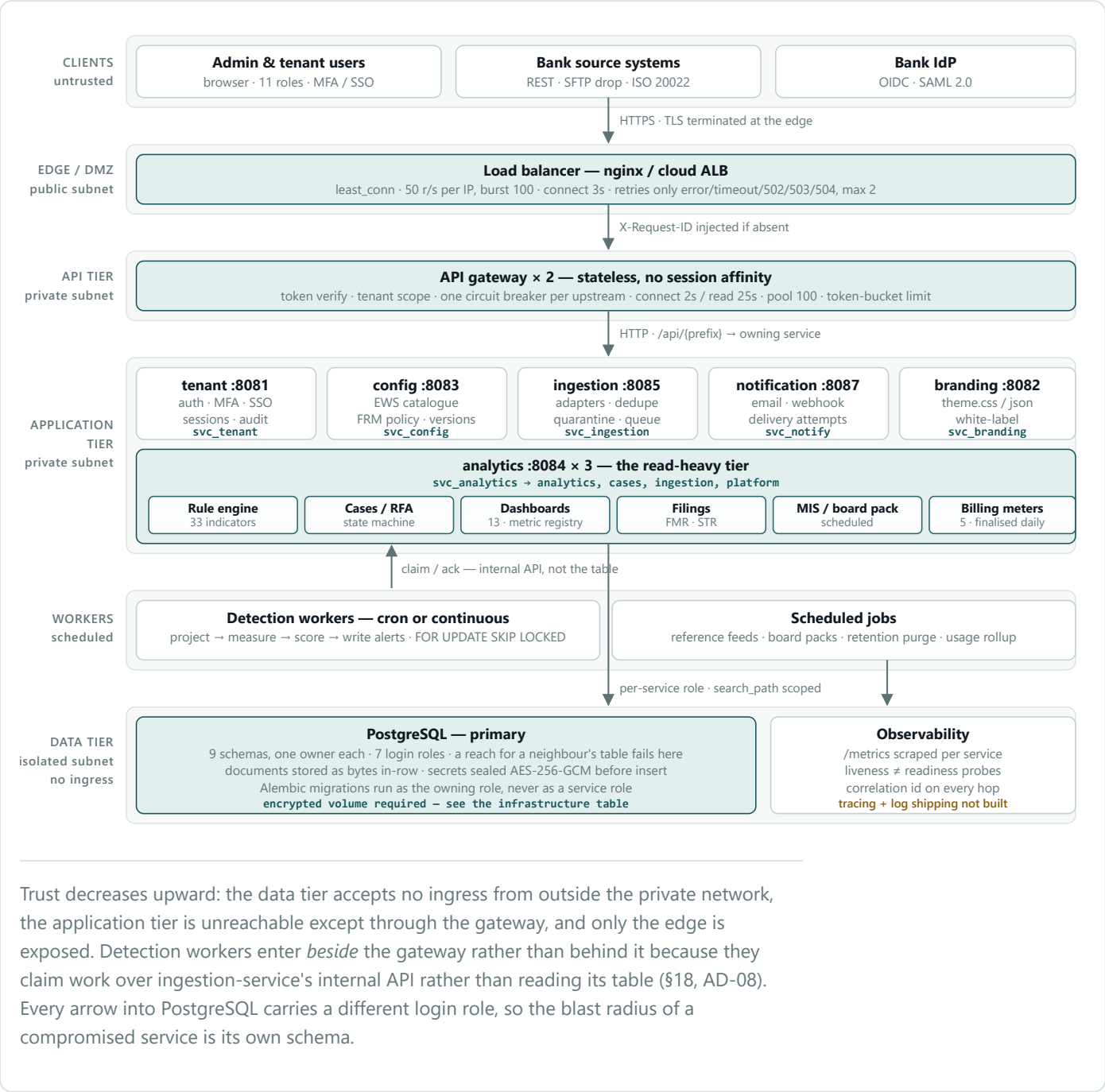
Five meters, aggregated per tenant per day and then marked final. A finalised day is never recomputed: an invoice that changes after it is issued is not an invoice. Seats are a high-water mark within the day rather than a sum — summing daily seat counts across a month bills a bank thirty times for the same analyst.

Tracing remains the notable absence. Correlation ids flow correctly across every service, but nothing collects them, so the cost has been paid and the benefit is not yet available.

18. Deployment architecture

How a production installation is laid out, what each hop is allowed to do, and the

infrastructure it needs underneath.



Service-to-service communication

There is no message bus. Every inter-service call is synchronous HTTP through the gateway, or a direct internal call guarded by a shared key; the `platform` schema plays the part a bus or Redis would (§17, AD-04).

CALLER	CALLEE	PATH / MECHANISM	GUARD
Browser, bank systems	Load balancer	HTTPS	50 r/s per IP, burst 100; TLS terminates here

CALLER	CALLEE	PATH / MECHANISM	GUARD
Load balancer	gateway	<code>least_conn</code> , keepalive 32	Retries only on error/timeout/502/503/504, at most twice — a non-idempotent write is never replayed
gateway	tenant · config · branding · analytics · ingestion · notification	<code>/api/{prefix}</code> → the owning service's own prefix	Token verified and tenant scope resolved <i>before</i> routing; one circuit breaker per upstream; connect 2 s, read 25 s; bounded pool
Detection workers	ingestion	Internal claim / ack API	<code>X-Internal-Key</code> ; <code>FOR UPDATE SKIP LOCKED</code> so replicas take different slices
analytics	tenant	Internal identity lookup for pack recipients	<code>X-Internal-Key</code> ; never exposed through the gateway
analytics	Reference sources	Outbound HTTPS pull	Credential sealed at rest; shrinkage check before a version is promoted
Every service	PostgreSQL	Its own login role and <code>search_path</code>	Cross-schema reach fails as <i>relation does not exist</i> , in the database
Every service	Config generation counter	<code>platform.config-generation</code> , read per call	A replica holding a stale generation discards its cache — this is why replicas cannot disagree

Cloud infrastructure required

Cloud-neutral, with the constraint that RBI data-localisation puts every component in an Indian region and every backup with it. Sizing is for a mid-sized installation and should be revisited against the load test that BR-909 still owes.

COMPONENT	WHAT IS NEEDED	WHY / SIZING NOTE
Container orchestration	Managed Kubernetes (EKS / AKS / GKE) or ECS. 3 nodes minimum for a real availability zone spread	Seven services plus workers. One image, per-service endpoint. Manifests exist for analytics-service only — the rest is outstanding
Managed PostgreSQL	16 vCPU / 64 GB class primary, gp3-equivalent storage, encryption at rest enabled , PITR, a standby in a second AZ	Operational <i>and</i> analytical store (§17, AD-02). Volume encryption is the deployment control the application cannot supply (BR-712). Documents live in-row, so storage grows with evidence, not just transactions
Load balancer	Cloud ALB or nginx ingress, TLS 1.2+, WAF in front	Terminates TLS, applies the per-IP limit. The in-process gateway limiter is per replica and is a safety valve, not a billing control
Secrets manager	KMS / Key Vault / Secret Manager holding <code>JWT_SECRET</code> , <code>INTERNAL_API_KEY</code> , <code>CP_ENCRYPTION_KEYS</code> , per-service DB passwords	Keys are versioned in the ciphertext, so rotation is additive. Custody must sit outside the database or a stolen backup is self-decrypting (§17, AD-10)
Object storage	S3 / Blob for database backups, exported bundles and spooled packs	Not on the request path today — case documents are in-row. This becomes a request-path dependency if that changes
SFTP endpoint	Managed SFTP (AWS Transfer or equivalent) writing to the intake bucket or volume	Banks that cannot open an API firewall drop files. Per-file checksum makes a re-drop a no-op

COMPONENT	WHAT IS NEEDED	WHY / SIZING NOTE
SMTP relay / webhook egress	SES / Graph / on-prem relay, plus outbound allow-listing	Notification delivery. Absent a relay the driver records the failure rather than claiming a send
Monitoring	Prometheus-compatible scrape, alerting on pipeline freshness, and a log sink	<code>/metrics</code> is exposed per service. Tracing and log shipping are <i>not built</i> — an incident today is investigated per host
Scheduler	Kubernetes CronJob or equivalent for detection, reference feeds, board packs, retention, usage rollup	Retention runs under the owning role; the others under their service role
Network	Three subnets — public edge, private application, isolated data with no route out. Private endpoints for managed services	Matches the trust bands in the figure. The data tier taking no ingress is what makes the per-service roles a second layer rather than the only one
CI/CD	Pipeline running the suite against a throwaway database, <code>alembic check</code> , then migrate-before-deploy	Migrations are a deploy step with different privileges from serving traffic
Backup / DR	PITR plus cross-region copy, and a <i>tested</i> restore runbook with stated RPO and RTO	Not yet done. An untested restore is an assumption, and the outsourcing directions expect the numbers evidenced (BR-908)

MULTI-TENANCY IS LOGICAL, AND THAT IS A COMMERCIAL DECISION AS MUCH AS A TECHNICAL ONE

All tenants share one database, isolated by tenant scope in every query and by schema ownership per service. A bank that refuses shared infrastructure needs a dedicated installation — which the architecture supports, since nothing assumes more than one tenant, but which changes the cost model and the DR design. That choice is open as BRD OD-01 and should be settled before the first contract, not after.

DEPLOYMENT ARTEFACT	STATE
Single container image, per-service endpoint	Built
Compose overlay with replicas, nginx and per-service DB roles	Built
Per-service database roles and schema grants	Built
Kubernetes manifests with rolling update and both probes	analytics-service only
Helm chart, ingress, HPA, PodDisruptionBudget	Pending
Secrets manager integration (keys read from env today)	Pending
Tested backup, restore and DR runbook	Pending

19. Decision record

Choices worth revisiting deliberately rather than by accident.

REF	DECISION	RATIONALE	REVISIT WHEN
AD-01	Python + FastAPI	Analytical and data-engineering ecosystem; fast to evolve rule logic	Sustained latency requirements make a JVM or Go runtime worth the rewrite
AD-02	PostgreSQL as both operational and analytical store	One correct store beats two inconsistent ones; exact integer arithmetic; window functions sufficient at current scale	Query latency degrades on multi-year data — a ClickHouse engine slots in behind the existing interface
AD-03	Schema-per-service in one instance, not database-per-service	Enforces ownership without five clusters to operate; migrations stay coordinated	A tenant demands physical separation, or a service needs independent scaling of its store
AD-04	Database-backed generation cache rather than Redis	Multi-replica correctness with no new infrastructure to run and monitor	Generation reads become hot enough to measure
AD-05	Rules engine rather than ML for detection	Explainable by construction; a bank can defend every alert to a regulator	False-positive load justifies the full FREE-AI model-governance obligation set
AD-06	Batch worker rather than stream processing	Correct and simple; adequate for near-real-time alerting	Interdiction is in scope, or latency budgets tighten below seconds
AD-07	Charting library vendored locally, no CDN	Several target banks deploy air-gapped	Never, unless all deployments gain outbound internet
AD-08	Detection reads the ingestion queue over an internal API	Keeps ownership with ingestion-service; substituting a queue changes two endpoints	Volume justifies Kafka or similar
AD-09	analytics-service keeps growing rather than splitting	Every candidate for extraction — filings, board packs, accountability — reads the metric registry and the case. Splitting now means either duplicating the registry or serving it over the network on the reconciliation path	OD-02 is decided, or one sub-domain's write load starts affecting dashboard latency
AD-10	Application-layer sealing of secrets, not column encryption in the database	Keeps key custody outside the database, so a database backup is not self-decrypting; key version travels with the ciphertext	The deployment gains an HSM or a managed KMS worth binding to

REF	DECISION	RATIONALE	REVISIT WHEN
AD-11	Simulation reuses the scoring engine with writes suppressed	A separate estimator is a second implementation of the rule semantics; when the two disagree nobody can say which is right	Never — this one is load-bearing for trust in retuning
AD-12	Prometheus text exposition written directly, no client library	Small stable format; the dependency would be carried into all seven services for a few dozen lines	Exemplars, native histograms or push-gateway semantics are needed
AD-13	Charting and diagram bundles vendored and digest-pinned	Air-gapped deployment, and 3.3 MB of third-party JavaScript is inlined into documents that go to a regulator. A substituted bundle stops the build	Never, unless all deployments gain outbound internet
AD-14	AI/ML roadmap Phase 2 model scoring runs in-process inside the existing detection worker, not a new service; CPU-only gradient-boosted trees or an isolation forest, not deep learning	Reuses AD-01's Python/FastAPI stack and AD-09's "one implementation, not two" reasoning: scoring needs the same account context and feature computation the rules engine already builds per batch (§6). A network hop to a separate scoring service, or a second feature pipeline, buys nothing yet. Tabular velocity/anomaly detection is squarely gradient-boosted-trees' and isolation-forest's use case; no GPU, no separate model-serving stack (TorchServe, Triton) needed at this scale	The batch worker's latency budget stops holding at contracted volume (revisit alongside BR-909's load-test evidence), or Phase 3's GNN work needs heavier compute (embeddings, matrix ops) that does not fit the same process — that decision is Phase 3's to make, not retroactively Phase 2's

REF	DECISION	RATIONALE	REVISIT WHEN
AD-15	AI/ML roadmap Phase 3's GNN runs as a periodic scheduled script (<code>scripts/refresh_ring_scores.py</code> , the same shape as <code>run_subscriptions.py</code> / <code>run_retention.py</code>), writing <code>analytics.account_ring_score</code> , which the detection worker only reads — never a live forward pass in the payment path. PyTorch + PyTorch Geometric (CPU-only), recorded separately from AD-14	A GraphSAGE pass over a whole tenant graph does not fit AD-14's per-transaction placement without making every batch as slow as a retrain (AD-06's batch-not-stream reasoning, applied one level up). PyTorch Geometric is still a materially heavier dependency than AD-14's scikit-learn (torch's CPU wheel alone is ~125MB), recorded on its own so a future audit of "why does this deployment carry PyTorch" doesn't have to reverse-engineer it from AD-14's unrelated reasoning. The compiled-extension risk this decision flagged did not materialise: PyG 2.8's pure-PyTorch message-passing fallback covers <code>SAGEConv</code> without <code>torch-scatter</code> / <code>torch-sparse</code> at all - both install and a working forward pass were verified on this deployment's own target platform (Windows, CPU) before this phase was built, not assumed	Refresh cadence or embedding cost make the scheduled-script placement itself wrong (a managed feature-store or streaming graph library becomes justified), or a lighter graph-feature approach (handcrafted centrality/clustering measures scored by AD-14's existing pipeline, no PyTorch at all) turns out to catch the same rings without the dependency weight - this was considered and not chosen, on the reasoning that "GNN-based fraud-ring detection" is what the BRD names, and handcrafted centrality features scored by a tree model is a different, smaller claim than that

20. Known gaps

Architectural debt stated plainly, with the consequence of leaving it. Gates refer to the release plan in BRD v1.9 §17.

CLOSED SINCE V1.0

V1.0 GAP	NOW
No MFA, SSO, session management or lockout	Closed §13
No notification channel	Closed notification-service
No encryption-at-rest configuration or retention/purge	Application layer closed §14; volume encryption outstanding
No FMR or STR artefact generation	Assembly closed §10; prescribed format outstanding
Eight indicators inert for want of reference feeds	Five remain §7 — and they are reported unmeasurable, not scored clean

V1.0 GAP	NOW
No file/SFTP intake, backfill or replay	Closed §6, §8 — plus ISO 20022 and CBS events
No metrics	Closed §16
No inline / real-time decisioning path	Closed §8 — Lane A, in shadow mode
No machine credential; intake needed a human account	Closed §13
Suspension gated the console login only	Closed §13 — the machine path is gated too
Exit export returned configuration but not the fraud record	Closed §14 — and it now declares its own completeness
No console UI for identity-provider configuration	Closed §14 — tenant administrators create, edit and delete their own OIDC/SAML providers; role mapping is validated against the tenant's own assignable roles, and a client secret is write-only, never echoed back
No console or API path to configure a tenant's FRM policy	Closed §11 — policy is now an accepted config kind; a tenant admin can create and activate a policy version, including the PMLA Principal Officer designation, gated by the same board-attestation requirement as before
Inline decision policy is not yet in config-service versioning	Closed — decision_policy is now an accepted config kind. A rail cannot be marked enforced without a per-rail graduation record (observation window, false-positive rate, board/ACB approving body) attached to the same version, checked before the version can even be proposed; a second, different eligible person must still confirm activation (BR-715) before decision-service's cached rail policy actually changes

OPEN

GAP	CONSEQUENCE IF UNADDRESSED	GATE
No full-disk or tablespace encryption	Application-layer sealing protects a database reader; it does not protect a stolen disk or a discarded backup. An IS review asks for both	R1
FMR and STR format bindings declared not_bound	The return is assembled, validated, hashed and retained, but the platform cannot claim it is in the prescribed format. Blocked on OD-04, not on engineering. The STR path's own defect — the Principal Officer field had no source at all — is fixed and verified end-to-end against a live case	R2
Capacity evidence is single-worker only	Detection latency is now measured — 200 tps per worker at p99 2.65 s, with the harness refusing to call a run sustained if the backlog grows across it. What is still unevidenced is multi-worker scaling and the managed-database deployment, so a contracted peak above one worker's throughput cannot yet be underwritten	R4
Five indicators still unmeasurable — SME-03, CHN-02, TBM-01/02/03	Correctly reported as unmeasurable, but it is EWS coverage a bank is paying for and not receiving. No invoice, geolocation or trade-finance feed is contracted	R3
No distributed tracing or log shipping	Metrics, structured logs and a request id that crosses every service exist; nothing ships them anywhere. An incident is investigated by reading files on each host	R4

GAP	CONSEQUENCE IF UNADDRESSED	GATE
No tested backup, restore or DR runbook	RPO and RTO are aspirations. An untested restore is an assumption	R4
No right-to-audit evidence pack or sub-processor register	The outsourcing directions give the bank and RBI a right to audit the service provider. It has to be answerable on request	R4
Kubernetes manifests for one service only	Not deployable to a managed cluster as-is	R4
Lane A ships in shadow mode; no tenant enforces yet	Every inline decision is computed and recorded, but the enforced answer is always <code>allow</code> . The <i>how</i> of graduating a rail to enforcement is now built — a per-rail graduation record, board sign-off and maker-checker confirmation. What's still open is narrower: no tenant has actually exercised it, so the false-positive cost of declining a real payment remains modelled rather than observed — and that number, not the latency, is what a bank will negotiate over	R3
Lane A has no per-pair set store	Beneficiary-age and counterparty-novelty indicators (CPT-01, VEL-02) need a per-account-pair set, not a counter. They are correctly reported as skipped inline rather than approximated, but two of the more useful inline signals are therefore unavailable in the payment window	R3
Case management not a separate deployable	Write-heavy and read-heavy workloads scale together; acceptable today. See AD-09	OD-02

21. AI/ML roadmap — architecture

How the four phases this document's own [AD-05](#) ("rules engine rather than ML — revisit when...") anticipates would actually be built, against the architecture as it stands today — not a restatement of the business case, which lives at BRD [§19](#). None of this is built; each phase's design below only firms up once the phase before it lands.

PHASE 1 — DEVICE & BEHAVIOURAL BIOMETRICS

No new service. Ingestion-service's transaction payload already carries a device id (§5); this phase widens that same payload with a device-posture and behavioural-biometric feature vector, sourced from VideoPD / the Human Fraud Detection Framework rather than captured here, and adds new sub-rules to the channel/device (CHN) family the detection runtime already evaluates (§6) — CHN-01 and CHN-03 are measured from transactions alone today; the new sub-rules join them at the same evaluation step, not a separate one. No architectural decision is revisited: AD-05 (rules engine, not ML) still holds, because this phase adds signal inputs to the existing rules engine rather than a scored model.

PHASE 2 — TRANSACTION VELOCITY & ANOMALY-DETECTION MODELS **BUILT**

The first phase that touches AD-05 directly, and the first one actually built. The `model` config kind and its champion/challenger gate (BR-807/808) were already built in config-service; this phase added the scoring side — a model-scoring step in the detection worker's sequence (§6) that runs alongside rule evaluation rather than replacing it (in the same loop that already merges rule observations and `cbs_features.observe_loan()`'s loan-conduct scores), reading the active `model`

version the same way the worker already reads the active rule catalogue (`rules.active_model()`), mirroring `active_rules()` and `policy()`'s own cached-fetch shape).

Runtime — AD-14, as decided. In-process inside analytics-service, not a separate scoring service behind AD-08's internal-API pattern: scoring needs the same account context `features.load_context` already builds per batch, and a second process would mean either duplicating that computation or paying a network hop per transaction for every scored row — the same reasoning AD-09 already applies to keeping analytics-service one service. CPU-only, an `sklearn.ensemble.IsolationForest` over five tabular features (velocity, counterparty, outflow-ratio, small-credit and structuring signals already computed for VEL-01/LAY-01/LAY-02/VEL-03/SME-01); Python + FastAPI (AD-01) already carries the dependencies this needs, no GPU or separate model-serving stack. `services/analytics_service/app/detection/model_scoring.py`.

Calibration, not a guessed constant. `score_samples` is lower-is-more-abnormal with no fixed range; scoring the mid-point of a real tenant's own traffic against a hand-picked transform put the bulk of ordinary traffic above any sane threshold during development. The trained artifact instead carries its own training set's 1st/99th-percentile score bounds, and `model_scoring.score()` rescales linearly against them, clamped to [0,1] — a live transaction outside the training range becomes a boundary value, not a value outside the catalogue's own scale.

Built: the `model` config body's `model_artifact` slot (format, object-storage `uri`, `sha256`) sits alongside `challenger_evaluation` and the committee attestation, validated by the same route (`config_service/app/model_artifact.py`) at proposal time — refused before `pending_activation`, the same discipline the other two gates already had. Only `sklearn-joblib` is a supported format; `onnx` is named as a future option and explicitly refused, not silently accepted and left to fail unexplained at scoring time. Only `file://` URIs are implemented today — object storage (`s3://`, azure blob) is where a real deployment's artifacts belong, not built because nothing in this environment has that infrastructure to point at. A batch loads the artifact once (cached by URI+sha256, so a replacement file is loaded fresh rather than silently reusing a stale one) and verifies its digest before every deserialisation, refusing a mutated or truncated file rather than scoring with something nobody approved. See `scripts/train_velocity_anomaly_model.py` for how a version is actually produced — an unsupervised `IsolationForest` fit against whatever history a tenant already has, since no tenant has labelled fraud outcomes to train against yet. **This is a first working fit, not a validated model** — detection quality against real fraud outcomes is unmeasured until BR-806's backtesting harness runs against live traffic, which this phase does not claim to have done.

PHASE 3 — GNN-BASED FRAUD-RING DETECTION **BUILT**

The account graph `GET /analytics/{tid}/graph` already computes (built for the Investigator/Account 360/AML network-graph view) was the natural training input — no new data model, a new consumer of data that already existed, within one tenant's own graph. Training needs the tenant's *whole* transaction graph, not the case- or account-scoped neighbourhood the live endpoint returns for display (or its unscoped "busiest structure" fallback, itself limited to already-LAY-flagged traffic) — built as its own query variant, `graph_model.build_tenant_graph()`, not a new data source.

Resolved by OD-07: this phase did not wait on a cross-tenant data-sharing decision, because it did not need one — the isolation decisions this platform had already made

(AD-03: schema-per-service in one instance; the multi-tenancy model in §9: no cross-tenant queries, enforced at the database role) are simply unaffected by training a model on data one tenant already owns.

Task and model. No ring labels exist to train against, same situation Phase 2 was in — so this is self-supervised, not classification: a 2-layer **GraphSAGE**

(`graph_model.RingScoreEncoder` ; inductive, deliberately not a transductive GCN — an account graph gains new nodes constantly, and an inductive model scores one it has never seen without retraining) produces a node embedding per account, seeded with Phase 2's own five features plus three the GNN specifically needed and Phase 2 never did — log-degree and log in/out-flow, since a model with no signal about neighbourhood size or flow cannot use its own aggregation for anything. Trained on link prediction (a real edge's endpoints should dot-product higher than a random pair's, via negative sampling) — a standard, label-free way to make an embedding space reflect graph structure. Those embeddings are then scored for anomalousness by the *same* calibrated-percentile pipeline Phase 2 built (`model_scoring.load()` / `score()`), extended (not replaced) to reconstruct the encoder from the bundle first — an embedding is just a richer feature vector to the scoring half, so that code needed no new logic, only a new upstream feature computation. Surfaced as **LAY-05** (fraud-ring membership) — the layering/mule family, not VEL, since this is topology, not velocity.

Runtime — does not reuse AD-14's placement, by design. A GraphSAGE forward pass over a whole tenant graph is not a per-transaction operation the way Phase 2's five-feature IsolationForest call is; recomputing embeddings for the whole graph on every batch the detection worker claims would make every batch as slow as a full retrain. This phase runs as a periodic scheduled script instead

(`scripts/refresh_ring_scores.py` , the same shape `scripts/run_subscriptions.py --once` and `scripts/run_retention.py` already are, driven by cron/Task Scheduler, not a new microservice — AD-06's reasoning: batch, not stream) that refreshes `analytics.account_ring_score` (tenant, account, score, model version, computed at; upserted, one row per account) on whatever cadence an operator schedules it at. The detection worker's per-transaction loop does a cheap primary-key lookup against that table for LAY-05, the same shape `device_clusters()` 's per-batch dict-return already has for LAY-04 — never a live GNN forward pass in the payment path. An account absent from the table (new since the last refresh, or the refresh has never run) contributes nothing to that transaction's observations — correctly not the same as a confirmed-typical score of zero, and not the same failure as no model existing at all: the coverage report distinguishes "no active graph-ring-score model for this tenant" (LAY-05 genuinely unmeasurable) from "this specific account has no score yet" (silently absent from one transaction's observations, exactly like an unprofiled account's CHN-03).

Dependency, recorded separately — AD-15. PyTorch + PyTorch Geometric (CPU-only) is a materially heavier dependency than AD-14's scikit-learn/joblib/numpy (torch's own CPU wheel is ~125MB) — deliberately not folded into AD-14's "no GPU, no separate model-serving stack" framing, since Phase 2's choice does not carry over just because both phases are ML, exactly what AD-14 itself said Phase 3 should not assume. The compiled-extension risk flagged when this was scoped (`torch-scatter` / `torch-sparse` , historically the fragile half of a PyG install) did not materialise: PyG 2.8's pure-PyTorch message-passing fallback covers `SAGEConv` without either, verified with a real install and a real forward pass on this deployment's own target platform before the rest of this phase was built.

The prerequisite this phase exposed is fixed, not merely documented. Phase 2's `/internal/model/{tid}` and `rules.active_model()` used to return the tenant's first active `model`-kind row, full stop — correct only while exactly one model existed. Both are now `/internal/model/{tid}/{name}` and `rules.active_model(tenant_id, name)`, keyed by the config's own `name` (read from `features.NEEDS_MODEL`, never retyped as a literal at a call site) — the config data model already supported two same-kind, differently-named active versions (`ConfigRepository.activate()` archives a sibling by tenant+kind+name, not tenant+kind alone); only the query layer had never been asked to distinguish them. `velocity-anomaly` (VEL-04) and `graph-ring-score` (LAY-05) are now active independently per tenant, each reachable only by its own name — verified by a test that activates one and confirms the other's name still reports unavailable.

PHASE 3B — FEDERATED LEARNING (DEFERRED)

Decoupled from Phase 3 by [OD-07](#). Federated learning's entire premise is training across a data boundary wider than one tenant — training within one tenant's own data is not federation, it is centralised training with an extra network hop, since there is only one data holder. That wider boundary is a materially different posture than anything AD-03 or §9's isolation model contemplates, and this document does not carry a design for it: no architecture is specified here because none is owed until a specific multi-bank engagement creates both the demand and the consent basis for it. [§13](#) (identity and access) and §9's isolation model are unaffected in the meantime — there is nothing to build against them.

PHASE 4 — EXPLAINABLE-AI DASHBOARDS & REGULATOR-READY REPORTING

No new service. Extends analytics-service's existing model-inventory and override-log endpoints (§12) with a per-alert rule-vs-model attribution field, and reuses the scheduled-distribution machinery already built for board packs (§7b, BR-507) to add an explainability appendix. Reporting only; there is no new data to protect or reconcile that §8's sixteen invariants don't already cover, since this phase reports on Phase 2/3's outputs rather than producing its own facts.

Verification basis. Statements about implemented behaviour in this document correspond to a build with 1078 passing automated tests, clean schema-migration drift, and sixteen reconciliation invariants passing across three tenants with live and demonstration data held side by side. Where this document says a control is enforced, the enforcement has been checked by reintroducing the fault and confirming the test fails — a guard that has never been seen to refuse anything is an assumption.

Read with BRD v1.9, which carries the numbered requirements, the release gates and the regulatory traceability this design serves.